Praktikum JavaScript & TypeScript

Modul 1 — Variabel, Tipe Data, dan Kondisi

1. Tujuan Praktikum

Setelah mengikuti praktikum ini, peserta mampu:

- Menjalankan file JavaScript dengan Node.js
- Memilih `const`, `let`, atau `var` dan menjelaskan alasannya
- Mengenal tipe data dasar dan perilaku angka di JavaScript
- Membedakan `===` dengan `==`, serta `??` dengan `||`
- Menulis percabangan dengan `if`, `switch`, dan ternary
- Menyusun urutan pengecekan kondisi yang benar

2. Dasar Teori

2.1. Menjalankan JavaScript

JavaScript bisa dijalankan di browser atau di Node.js. Di praktikum ini kita memakai Node.js. Simpan kode di file `.js`, lalu jalankan dari terminal.

```
const nama = "Dion";
console.log("Halo", nama);          // Halo Dion
console.log(`Halo ${nama}!`);       // Halo Dion!
```

Listing 1: halo.js

```
node halo.js
```

Tanda backtick (`` ` ``) membuat *template literal*. Ekspresi di dalam `${...}` dihitung lalu disisipkan ke string. Cara ini lebih mudah dibaca daripada menyambung string dengan `+`.

Di topik ini kita juga memakai fungsi sebagai "wadah" logika, karena soal meminta fungsi yang menerima input dan mengembalikan hasil. Sintaksnya mirip C, tetapi tanpa tipe data. Detail fungsi dibahas di topik 2.

```
function luasPersegi(sisi) {
  return sisi * sisi;
}
console.log(luasPersegi(4)); // 16
```

Listing 2: fungsi paling sederhana

2.2. Variabel: const, let, var

Keyword	Boleh diisi ulang?	Scope
`const`	Tidak	Block `{ }`
`let`	Ya	Block `{ }`
`var`	Ya	Function (bocor keluar block)

Aturan praktis: pakai `const` sebagai default. Ganti ke `let` hanya jika nilainya memang perlu diisi ulang, misalnya penghitung. Jangan pakai `var` di kode baru.

```
const tarifKamar = 500000;
tarifKamar = 600000;   // TypeError: Assignment to constant variable.

let antrean = 0;
antrean = antrean + 1; // boleh
```

Listing 3: const vs let

> **Perhatian**
> `const` melarang **mengisi ulang variabel**, bukan melarang **mengubah isi object**. Variabel `const` yang berisi object tetap menunjuk object yang sama, tetapi isi object itu boleh berubah.
>
> ```
> const pasien = { nama: "Budi", umur: 40 };
> pasien.umur = 41;            // boleh: isi object berubah
> pasien = { nama: "Ani" };    // TypeError: menunjuk object lain
> ```

Mengapa `var` dihindari? `var` tidak mengenal block. Variabel yang dibuat di dalam `if` atau `for` tetap terlihat di luar block, sehingga mudah tertimpa tanpa sengaja.

```
if (true) {
  var a = 1;
  let b = 2;
}
console.log(a); // 1
console.log(b); // ReferenceError: b is not defined
```

Listing 4: var bocor keluar block

> **Catatan**
> Interviewer sering bertanya "kenapa pakai `const`?". Jawaban yang baik: `const` memberi tahu pembaca bahwa nilai ini tidak akan diganti, sehingga kode lebih mudah dilacak. Jika nanti perlu diganti, ubah ke `let` dengan sadar.

2.3. Tipe Data

Tipe data primitif yang paling sering dipakai:

- `number`: bilangan bulat dan desimal memakai **satu** tipe yang sama. Tidak ada `int` atau `float` seperti di C.
- `string`: teks, ditulis dengan `"..."`, `'...'`, atau `` `...` ``.
- `boolean`: `true` atau `false`.
- `undefined`: variabel ada tetapi belum diberi nilai.
- `null`: sengaja dikosongkan oleh programmer.

Selain itu ada `object` (termasuk array), dibahas di topik 3 dan 4. Operator `typeof` mengembalikan nama tipe sebagai string.

```
console.log(typeof 42);        // "number"
console.log(typeof 3.14);      // "number"
console.log(typeof "42");      // "string"
console.log(typeof undefined); // "undefined"
console.log(typeof null);      // "object" (bug lama)
```

Listing 5: typeof

> **Perhatian**
> Angka desimal di JavaScript tidak selalu tepat: `0.1 + 0.2` menghasilkan `0.30000000000000004`. Untuk uang, simpan nominal sebagai bilangan bulat (Rupiah atau sen), bukan desimal.

Untuk pembulatan, pakai fungsi bawaan `Math`:

- `Math.round(2.5)` hasilnya `3` (pembulatan biasa)
- `Math.floor(2.9)` hasilnya `2` (ke bawah)

- `Math.ceil(2.1)` hasilnya `3` (ke atas)

Untuk mengecek apakah sebuah nilai adalah bilangan bulat, pakai `Number.isInteger(x)`.

2.4. Operator

Operator aritmetika: `+ - * /`, `%` (sisa bagi), dan `**` (pangkat). Operator logika: `&&` (dan), `||` (atau), `!` (bukan).

2.4.1. Perbandingan: `===` vs `==`

`===` (strict equality) membandingkan nilai **dan** tipe. `==` (loose equality) mengubah tipe dulu sebelum membandingkan, dan aturannya sering mengejutkan.

```
console.log(5 === "5");   // false (number vs string)
console.log(5 == "5");    // true  (string diubah jadi number)
console.log(0 == "");     // true
console.log(0 === "");    // false
```

Listing 6: === vs ==

Selalu pakai `===` dan `!==`. Pasangan `==` dan `!=` hampir tidak pernah dibutuhkan.

2.4.2. Truthy dan falsy

Di dalam `if`, nilai apa pun diubah menjadi boolean. Hanya enam nilai berikut yang dianggap `false` (falsy): `false`, `0`, `""`, `null`, `undefined`, dan `NaN`. Semua nilai lain truthy, termasuk `"0"`, `"false"`, dan array kosong `[]`.

2.4.3. Nilai default: `||` vs `??`

`a || b` menghasilkan `b` jika `a` falsy. `a ?? b` (nullish coalescing) menghasilkan `b` hanya jika `a` bernilai `null` atau `undefined`.

```
const stok = 0;
console.log(stok || "kosong"); // "kosong"  <- 0 dianggap
falsy
console.log(stok ?? "kosong"); // 0         <- 0 tetap dipakai
```

Listing 7: || vs ??

> **Perhatian**
> Jika `0` atau `""` adalah nilai yang sah (stok 0, diskon 0,

> catatan kosong), jangan pakai `||` untuk nilai default. Pakai `??`.

2.5. Percabangan

2.5.1. if, else if, else

Kondisi dicek dari atas ke bawah. Blok pertama yang kondisinya benar dijalankan, sisanya dilewati.

```
function kategoriUmur(umur) {
  if (umur < 0) {
    return "INVALID";
  } else if (umur < 13) {
    return "Anak";
  } else if (umur < 60) {
    return "Dewasa";
  } else {
    return "Lansia";
  }
}
```

Listing 8: kategori umur pasien

> **Perhatian**
> Urutan kondisi menentukan hasil. Jika kondisi yang lebih umum ditulis lebih dulu, kondisi yang lebih khusus di bawahnya tidak pernah tercapai. Pada Listing di atas, `umur < 60` aman ditulis setelah `umur < 13` karena anak-anak sudah “tersaring” lebih dulu.

2.5.2. Early return (guard clause)

Daripada membuat `if` bersarang, tangani kasus gagal lebih dulu lalu langsung `return`. Sisa fungsi hanya berisi jalur normal.

```
function bisaDaftar(umur, punyaKTP) {
  if (typeof umur !== "number") return "Umur harus angka";
  if (umur < 17) return "Belum cukup umur";
  if (!punyaKTP) return "KTP wajib";
  return "OK";
}
```

Listing 9: guard clause

2.5.3. switch

`switch` cocok untuk mencocokkan satu nilai dengan banyak pilihan tetap. Pencocokan memakai `===`.

```
function namaPoli(kode) {
  switch (kode) {
    case "A":
      return "Poli Anak";
    case "G":
      return "Poli Gigi";
    default:
      return "Tidak dikenal";
  }
}
```

Listing 10: switch

> **Perhatian**
> Tanpa `return` atau `break`, eksekusi "jatuh" ke `case` berikutnya (*fallthrough*). Karena pencocokan memakai `===`, `switch` dengan `case "1"` tidak cocok dengan angka `1`.

2.5.4. Ternary

Ternary `kondisi ? nilaiJikaBenar : nilaiJikaSalah` adalah `if/else` yang menghasilkan nilai. Cocok untuk pilihan sederhana dua cabang.

```
const label = umur >= 60 ? "Lansia" : "Bukan lansia";
```

Listing 11: ternary

Hindari ternary bersarang. Jika butuh lebih dari dua cabang, pakai `if/else`.

> **Coba Sendiri**
> Buat file `coba1.js`. Sebelum menjalankan, **tulis tebakanmu** untuk setiap baris di laporan. Setelah itu jalankan, lalu bandingkan.
>
> ```
> const keranjang = ["sabun"];
> keranjang.push("sikat gigi");
> console.log(keranjang.length);
> ```

```
console.log("10" === 10, "10" == 10);
console.log(null ?? "default", 0 ?? "default", "" ||
"default");
console.log(Boolean("0"), Boolean(""), Boolean([]));
console.log(Math.round(4.5), Math.floor(-4.5),
Math.ceil(4.01));
console.log(0.1 + 0.2 === 0.3);

for (var i = 0; i < 3; i++) {}
console.log(i);
```

1. Tebakan mana yang salah? Jelaskan penyebab setiap tebakan yang salah.
2. Mengapa `keranjang.push(...)` tidak error padahal `keranjang` dibuat dengan `const`?
3. Ganti `var` di baris terakhir menjadi `let`. Apa yang terjadi, dan mengapa?

Sertakan screenshot hasil tiap langkah.